

Python Tip: Iterators

Having memory issues storing large datasets in lists? Instead of storing everything in memory, use iterators.

Lists vs Iterators

Iterators work just like lists. You can create them using comprehensions, with parentheses instead of brackets.

```
# Create lists and iterators using comprehensions
mylist = [x for x in range(10)] # [] for list
myiter = (x for x in range(10)) # () for iterator
```

```
# Loop over them the same way
for i in mylist:
    print(i, end=" ")
print()

for i in myiter:
    print(i, end=" ")
```

Memory Efficiency

Iterators are more memory-efficient than lists because they only store the current element in memory.

```
# This stores 1,000,000 integers in memory
mylist = [x for x in range(1_000_000)]

# This stores only the current integer in memory
myiter = (x for x in range(1_000_000))

print(f"List created with {len(mylist)} elements")
print(f"Iterator created")
```

```
List created with 1000000 elements
Iterator created
```

Using `next()` with Iterators

The catch is that iterators do not accept indexing. Use the `next()` function to get the next element.

```
mylist = [x for x in range(10)]
myiter = (x for x in range(10))

# This works for lists
x2 = mylist[2]
print(f"Third element of list: {x2}")

# But myiter[2] would raise an error!
# Use next() instead
x0 = next(myiter) # Get the first element
x1 = next(myiter) # Get the second element
x2 = next(myiter) # Get the third element
print(f"Third element of iterator: {x2}")
```

```
Third element of list: 2
Third element of iterator: 2
```


Creating Custom Iterators

For complex iterators like image loading, make iterators using classes by defining `__init__`, `__iter__`, and `__next__`.

Raise `StopIteration` to stop the iteration.

```
from PIL import Image

class MyImages:
    def __init__(self): # Specify initial values
        self.index = 0
        self.max = 100

    def __iter__(self): # Required for iteration
        return self

    def __next__(self): # Specify what to return
        if self.index >= self.max:
            raise StopIteration
        img = Image.open(f"image_{self.index}.png")
        self.index += 1
        return img

# This will load images one at a time
for img in MyImages():
    img.show()
```

Wrap-Up

Now you can use iterators to load large datasets one element at a time, saving memory.

Follow me for more tips.

Shep Bryan IV